1. Micropython 快速入门

1. Micropython 快速入门
    1.1 什么是 Micropython
    1.2 Python3 基础知识
        注释
        原始数据类型和运算符
        变量和集合
        流程控制和迭代器
        函数
        模块
        类
            类的继承
    1.3 Micropython其它相关API用法

1.1 什么是 Micropython

MicroPython 是编程语言 Python3 的精简高效实现，语法和 Python3 保持一致，但只实现了 Python 标准库的一小部分。MicroPython经过优化，适用于物联网 (IoT)、消费电子和嵌入式系统等领域。python是一种弱类型的语言，语法高效简洁，在开始我们的K230使用之前，我们先来掌握一些基础的python语法知识

1.2 Python3 基础知识

本文内容基于 Louie Dinh 的《10 分钟快速熟悉Python3》，由 Geoff Liu 翻译

注释

```
# 用井字符开头的是单行注释

""" 多行字符串用三个引号
    包裹，也常被用来做多
    行注释
"""
```

原始数据类型和运算符

```
####################################################
## 1. 原始数据类型和运算符
####################################################

# 整数
3  # => 3

# 算术没有什么出乎意料的
1 + 1  # => 2
8 - 1  # => 7
10 * 2  # => 20

# 但是除法例外，会自动转换成浮点数
```

```
35 / 5  # => 7.0
5 / 3  # => 1.6666666666666667

# 整数除法的结果都是向下取整
5 // 3     # => 1
5.0 // 3.0 # => 1.0 # 浮点数也可以
-5 // 3  # => -2
-5.0 // 3.0 # => -2.0

# 浮点数的运算结果也是浮点数
3 * 2.0 # => 6.0

# 模除
7 % 3 # => 1

# x的y次方
2**4 # => 16

# 用括号决定优先级
(1 + 3) * 2  # => 8

# 布尔值
True
False

# 用not取非
not True  # => False
not False  # => True

# 逻辑运算符，注意and和or都是小写
True and False # => False
False or True # => True

# 整数也可以当作布尔值
0 and 2 # => 0
-5 or 0 # => -5
0 == False # => True
2 == True # => False
1 == True # => True

# 用==判断相等
1 == 1  # => True
2 == 1  # => False

# 用!=判断不等
1 != 1  # => False
2 != 1  # => True

# 比较大小
1 < 10  # => True
1 > 10  # => False
2 <= 2  # => True
2 >= 2  # => True

# 大小比较可以连起来!
1 < 2 < 3  # => True
2 < 3 < 2  # => False
```

```
# 字符串用单引双引都可以
"这是个字符串"
'这也是个字符串'

# 用加号连接字符串
"Hello " + "world!"  # => "Hello world!"

# 字符串可以被当作字符列表
"This is a string"[0]  # => 'T'

# 用.format来格式化字符串
"{} can be {}".format("strings", "interpolated")

# 可以重复参数以节省时间
"{0} be nimble, {0} be quick, {0} jump over the {1}".format("Jack", "candle
stick")
# => "Jack be nimble, Jack be quick, Jack jump over the candle stick"

# 如果不想数参数，可以用关键字
"{name} wants to eat {food}".format(name="Bob", food="lasagna")
# => "Bob wants to eat lasagna"

# 如果你的Python3程序也要在Python2.5以下环境运行，也可以用老式的格式化语法
"%s can be %s the %s way" % ("strings", "interpolated", "old")

# None是一个对象
None  # => None

# 当与None进行比较时不要用 ==，要用is。is是用来比较两个变量是否指向同一个对象。
"etc" is None  # => False
None is None  # => True

# None，0，空字符串，空列表，空字典都算是False
# 所有其他值都是True
bool(0)  # => False
bool("")  # => False
bool([]) # => False
bool({}) # => False
```

变量和集合

```


####################################################
## 2. 变量和集合
####################################################

# print是内置的打印函数
print("I'm Python. Nice to meet you!")

# 在给变量赋值前不用提前声明
# 传统的变量命名是小写，用下划线分隔单词
some_var = 5
some_var  # => 5

# 访问未赋值的变量会抛出异常
```

```
# 参考流程控制一段来学习异常处理
some_unknown_var  # 抛出NameError

# 用列表(list)储存序列
li = []
# 创建列表时也可以同时赋给元素
other_li = [4, 5, 6]

# 用append在列表最后追加元素
li.append(1)    # li现在是[1]
li.append(2)    # li现在是[1, 2]
li.append(4)    # li现在是[1, 2, 4]
li.append(3)    # li现在是[1, 2, 4, 3]
# 用pop从列表尾部删除
li.pop()        # => 3 且li现在是[1, 2, 4]
# 把3再放回去
li.append(3)    # li变回[1, 2, 4, 3]

# 列表存取跟数组一样
li[0]  # => 1
# 取出最后一个元素
li[-1]  # => 3

# 越界存取会造成IndexError
li[4]  # 抛出IndexError

# 列表有切割语法
li[1:3]  # => [2, 4]
# 取尾
li[2:]  # => [4, 3]
# 取头
li[:3]  # => [1, 2, 4]
# 隔一个取一个
li[::2]   # =>[1, 4]
# 倒排列表
li[::-1]   # => [3, 4, 2, 1]
# 可以用三个参数的任何组合来构建切割
# li[始:终:步伐]

# 用del删除任何一个元素
del li[2]   # li is now [1, 2, 3]

# 列表可以相加
# 注意：li和other_li的值都不变
li + other_li   # => [1, 2, 3, 4, 5, 6]

# 用extend拼接列表
li.extend(other_li)   # li现在是[1, 2, 3, 4, 5, 6]

# 用in测试列表是否包含值
1 in li   # => True

# 用len取列表长度
len(li)   # => 6


# 元组是不可改变的序列
tup = (1, 2, 3)
```

```
tup[0]   # => 1
tup[0] = 3  # 抛出TypeError

# 列表允许的操作元组大都可以
len(tup)   # => 3
tup + (4, 5, 6)   # => (1, 2, 3, 4, 5, 6)
tup[:2]   # => (1, 2)
2 in tup   # => True

# 可以把元组合列表解包，赋值给变量
a, b, c = (1, 2, 3)     # 现在a是1，b是2，c是3
# 元组周围的括号是可以省略的
d, e, f = 4, 5, 6
# 交换两个变量的值就这么简单
e, d = d, e     # 现在d是5，e是4


# 用字典表达映射关系
empty_dict = {}
# 初始化的字典
filled_dict = {"one": 1, "two": 2, "three": 3}

# 用[]取值
filled_dict["one"]   # => 1


# 用 keys 获得所有的键。
# 因为 keys 返回一个可迭代对象，所以在这里把结果包在 list 里。我们下面会详细介绍可迭代。
# 注意：字典键的顺序是不定的，你得到的结果可能和以下不同。
list(filled_dict.keys())   # => ["three", "two", "one"]


# 用values获得所有的值。跟keys一样，要用list包起来，顺序也可能不同。
list(filled_dict.values())   # => [3, 2, 1]


# 用in测试一个字典是否包含一个键
"one" in filled_dict   # => True
1 in filled_dict   # => False

# 访问不存在的键会导致KeyError
filled_dict["four"]   # KeyError

# 用get来避免KeyError
filled_dict.get("one")   # => 1
filled_dict.get("four")   # => None
# 当键不存在的时候get方法可以返回默认值
filled_dict.get("one", 4)   # => 1
filled_dict.get("four", 4)   # => 4

# setdefault方法只有当键不存在的时候插入新值
filled_dict.setdefault("five", 5)  # filled_dict["five"]设为5
filled_dict.setdefault("five", 6)  # filled_dict["five"]还是5

# 字典赋值
filled_dict.update({"four":4}) # => {"one": 1, "two": 2, "three": 3, "four": 4}
filled_dict["four"] = 4  # 另一种赋值方法
```

```
# 用del删除
del filled_dict["one"]  # 从filled_dict中把one删除


# 用set表达集合
empty_set = set()
# 初始化一个集合，语法跟字典相似。
some_set = {1, 1, 2, 2, 3, 4}   # some_set现在是{1, 2, 3, 4}

# 可以把集合赋值于变量
filled_set = some_set

# 为集合添加元素
filled_set.add(5)   # filled_set现在是{1, 2, 3, 4, 5}

# & 取交集
other_set = {3, 4, 5, 6}
filled_set & other_set   # => {3, 4, 5}

# | 取并集
filled_set | other_set   # => {1, 2, 3, 4, 5, 6}

# - 取补集
{1, 2, 3, 4} - {2, 3, 5}   # => {1, 4}

# in 测试集合是否包含元素
2 in filled_set   # => True
10 in filled_set   # => False
```

流程控制和迭代器

```
####################################################
## 3. 流程控制和迭代器
####################################################

# 先随便定义一个变量
some_var = 5

# 这是个if语句。注意缩进在Python里是有意义的
# 印出"some_var比10小"
if some_var > 10:
    print("some_var比10大")
elif some_var < 10:    # elif句是可选的
    print("some_var比10小")
else:                  # else也是可选的
    print("some_var就是10")


"""
用for循环语句遍历列表
打印:
    dog is a mammal
    cat is a mammal
    mouse is a mammal
"""
for animal in ["dog", "cat", "mouse"]:
```

```
    print("{} is a mammal".format(animal))

"""
"range(number)"返回数字列表从0到给的数字
打印:
    0
    1
    2
    3
"""
for i in range(4):
    print(i)

"""
while循环直到条件不满足
打印:
    0
    1
    2
    3
"""
x = 0
while x < 4:
    print(x)
    x += 1  # x = x + 1 的简写

# 用try/except块处理异常状况
try:
    # 用raise抛出异常
    raise IndexError("This is an index error")
except IndexError as e:
    pass    # pass是无操作，但是应该在这里处理错误
except (TypeError, NameError):
    pass    # 可以同时处理不同类的错误
else:   # else语句是可选的，必须在所有的except之后
    print("All good!")   # 只有当try运行完没有错误的时候这句才会运行


# Python提供一个叫做可迭代(iterable)的基本抽象。一个可迭代对象是可以被当作序列
# 的对象。比如说上面range返回的对象就是可迭代的。

filled_dict = {"one": 1, "two": 2, "three": 3}
our_iterable = filled_dict.keys()
print(our_iterable) # => dict_keys(['one', 'two', 'three'])，是一个实现可迭代接口的对
象

# 可迭代对象可以遍历
for i in our_iterable:
    print(i)    # 打印 one, two, three

# 但是不可以随机访问
our_iterable[1]  # 抛出TypeError

# 可迭代对象知道怎么生成迭代器
our_iterator = iter(our_iterable)

# 迭代器是一个可以记住遍历的位置的对象
# 用__next__可以取得下一个元素
```

```
our_iterator.__next__()  # => "one"

# 再一次调取__next__时会记得位置
our_iterator.__next__()  # => "two"
our_iterator.__next__()  # => "three"

# 当迭代器所有元素都取出后，会抛出StopIteration
our_iterator.__next__() # 抛出StopIteration

# 可以用list一次取出迭代器所有的元素
list(filled_dict.keys())  # => Returns ["one", "two", "three"]
```

函数

```
####################################################
## 4. 函数
####################################################

# 用def定义新函数
def add(x, y):
    print("x is {} and y is {}".format(x, y))
    return x + y    # 用return语句返回

# 调用函数
add(5, 6)   # => 印出"x is 5 and y is 6"并且返回11

# 也可以用关键字参数来调用函数
add(y=6, x=5)   # 关键字参数可以用任何顺序


# 我们可以定义一个可变参数函数
def varargs(*args):
    return args

varargs(1, 2, 3)   # => (1, 2, 3)


# 我们也可以定义一个关键字可变参数函数
def keyword_args(**kwargs):
    return kwargs

# 我们来看看结果是什么：
keyword_args(big="foot", loch="ness")   # => {"big": "foot", "loch": "ness"}


# 这两种可变参数可以混着用
def all_the_args(*args, **kwargs):
    print(args)
    print(kwargs)
"""
all_the_args(1, 2, a=3, b=4) prints:
    (1, 2)
    {"a": 3, "b": 4}
"""
```

```
# 调用可变参数函数时可以做跟上面相反的，用*展开序列，用**展开字典。
args = (1, 2, 3, 4)
kwargs = {"a": 3, "b": 4}
all_the_args(*args)   # 相当于 foo(1, 2, 3, 4)
all_the_args(**kwargs)   # 相当于 foo(a=3, b=4)
all_the_args(*args, **kwargs)   # 相当于 foo(1, 2, 3, 4, a=3, b=4)


# 函数作用域
x = 5

def setX(num):
    # 局部作用域的x和全局域的x是不同的
    x = num # => 43
    print (x) # => 43

def setGlobalX(num):
    global x
    print (x) # => 5
    x = num # 现在全局域的x被赋值
    print (x) # => 6

setX(43)
setGlobalX(6)

# 函数在Python是一等公民
def create_adder(x):
    def adder(y):
        return x + y
    return adder

add_10 = create_adder(10)
add_10(3)   # => 13

# 也有匿名函数
(lambda x: x > 2)(3)   # => True

# 内置的高阶函数
map(add_10, [1, 2, 3])   # => [11, 12, 13]
filter(lambda x: x > 5, [3, 4, 5, 6, 7])   # => [6, 7]

# 用列表推导式可以简化映射和过滤。列表推导式的返回值是另一个列表。
[add_10(i) for i in [1, 2, 3]]  # => [11, 12, 13]
[x for x in [3, 4, 5, 6, 7] if x > 5]   # => [6, 7]
```

模块

注意，micropython不能使用pip去下载第三方模块

```
####################################################
## 5. 模块
####################################################
```

```
# 用import导入模块
import math
print(math.sqrt(16))  # => 4.0

# 也可以从模块中导入个别值
from math import ceil, floor
print(ceil(3.7))  # => 4.0
print(floor(3.7))   # => 3.0

# 可以导入一个模块中所有值
# 警告：不建议这么做
from math import *

# 如此缩写模块名字
import math as m
math.sqrt(16) == m.sqrt(16)   # => True

# Python模块其实就是普通的Python文件。你可以自己写，然后导入，
# 模块的名字就是文件的名字。

# 你可以这样列出一个模块里所有的值
import math
dir(math)
```

类

```
####################################################
## 6. 类
####################################################


# 定义一个继承object的类
class Human(object):

    # 类属性，被所有此类的实例共用。
    species = "H. sapiens"

    # 构造方法，当实例被初始化时被调用。注意名字前后的双下划线，这是表明这个属
    # 性或方法对Python有特殊意义，但是允许用户自行定义。你自己取名时不应该用这
    # 种格式。
    def __init__(self, name):
        # Assign the argument to the instance's name attribute
        self.name = name

    # 实例方法，第一个参数总是self，就是这个实例对象
    def say(self, msg):
        return "{name}: {message}".format(name=self.name, message=msg)

    # 类方法，被所有此类的实例共用。第一个参数是这个类对象。
    @classmethod
    def get_species(cls):
        return cls.species
```

```
    # 静态方法。调用时没有实例或类的绑定。
    @staticmethod
    def grunt():
        return "*grunt*"


# 构造一个实例
i = Human(name="Yahboom")
print(i.say("hi"))     # 印出 "Yahboom: hi"

j = Human("xzt")
print(j.say("hello"))  # 印出 "xzt: hello"

# 调用一个类方法
i.get_species()   # => "H. sapiens"

# 改一个共用的类属性
Human.species = "H. neanderthalensis"
i.get_species()   # => "H. neanderthalensis"
j.get_species()   # => "H. neanderthalensis"

# 调用静态方法
Human.grunt()   # => "*grunt*"
```

类的继承

```
# 定义一个继承object的类
class Human(object):

    # 类属性，被所有此类的实例共用。
    species = "H. sapiens"

    # 构造方法，当实例被初始化时被调用。注意名字前后的双下划线，这是表明这个属
    # 性或方法对Python有特殊意义，但是允许用户自行定义。你自己取名时不应该用这
    # 种格式。
    def __init__(self, name):
        # Assign the argument to the instance's name attribute
        self.name = name

    # 实例方法，第一个参数总是self，就是这个实例对象
    def say(self, msg):
        return "{name}: {message}".format(name=self.name, message=msg)

    # 类方法，被所有此类的实例共用。第一个参数是这个类对象。
    @classmethod
    def get_species(cls):
        return cls.species

    # 静态方法。调用时没有实例或类的绑定。
    @staticmethod
    def grunt():
        return "*grunt*"
```

```python
# 构造一个实例
i = Human(name="Yahboom")
print(i.say("hi"))     # 印出 "Yahboom: hi"

j = Human("xzt")
print(j.say("hello"))  # 印出 "xzt: hello"

# 调用一个类方法
i.get_species()   # => "H. sapiens"

# 改一个共用的类属性
Human.species = "H. neanderthalensis"
i.get_species()   # => "H. neanderthalensis"
j.get_species()   # => "H. neanderthalensis"

# 调用静态方法
Human.grunt()   # => "*grunt*"
# 继承机制允许子类可以继承父类上的方法和变量。
# 我们可以把 Human 类作为一个基础类或者说叫做父类，
# 然后定义一个名为 Superhero 的子类来继承父类上的比如 "species"、 "name"、 "age" 的属性
# 和比如 "sing" 、"grunt" 这样的方法，同时，也可以定义它自己独有的属性

# 基于 Python 文件模块化的特点，你可以把这个类放在独立的文件中，比如说，human.py。

# 要从别的文件导入函数，需要使用以下的语句
# from "filename-without-extension" import "function-or-class"


# 指定父类作为类初始化的参数
class Superhero(Human):

    # 如果子类需要继承所有父类的定义，并且不需要做任何的修改，
    # 你可以直接使用 "pass" 关键字（并且不需要其他任何语句）
    # 但是在这个例子中会被注释掉，以用来生成不一样的子类。
    # pass

    # 子类可以重写父类定义的字段
    species = 'Superhuman'

    # 子类会自动的继承父类的构造函数包括它的参数，但同时，子类也可以新增额外的参数或者定义，
    # 甚至去覆盖父类的方法比如说构造函数。
    # 这个构造函数从父类 "Human" 上继承了 "name" 参数，同时又新增了 "superpower" 和
    # "movie" 参数:
    def __init__(self, name, movie=False,
                 superpowers=["super strength", "bulletproofing"]):

        # 新增额外类的参数
        self.fictional = True
        self.movie = movie
        # 注意可变的默认值，因为默认值是共享的
        self.superpowers = superpowers

        # "super" 函数让你可以访问父类中被子类重写的方法
        # 在这个例子中，被重写的是 __init__ 方法
        # 这个语句是用来运行父类的构造函数:
        super().__init__(name)

    # 重写父类中的 sing 方法
```

```
    def sing(self):
        return 'Dun, dun, DUN!'

    # 新增一个额外的方法
    def boast(self):
        for power in self.superpowers:
            print("I wield the power of {pow}!".format(pow=power))


if __name__ == '__main__':
    sup = Superhero(name="Tick")

    # 检查实例类型
    if isinstance(sup, Human):
        print('I am human')
    if type(sup) is Superhero:
        print('I am a superhero')

    # 调用父类的方法并且使用子类的属性
    print(sup.get_species())    # => Superhuman

    # 调用被重写的方法
    print(sup.sing())           # => Dun, dun, DUN!

    # 调用 Human 的方法
    sup.say('Spoon')            # => Tick: Spoon

    # 调用 Superhero 独有的方法
    sup.boast()                 # => I wield the power of super strength!
                                # => I wield the power of bulletproofing!

    # 继承类的字段
    sup.age = 31
    print(sup.age)              # => 31

    # Superhero 独有的字段
    print('Am I Oscar eligible? ' + str(sup.movie))
```

1.3 Micropython其它相关API用法

可以参考嘉楠科技（K230芯片研发公司）提供的API手册：

API 手册 — CanMV K230